

Section A: Introduction to NoSQL and MongoDB (10 Questions)

1. What are the main differences between SQL and NoSQL databases?

SQL:

SQL - relational, structured tables with fixed schema.

SQL uses JOINS, strict ACID transactions.

NoSQL:

NoSQL - non-relational, flexible schema (JSON-like).

NoSQL is schema-less, supports horizontal scaling and huge data.

2. Explain the CAP theorem in the context of NoSQL databases.

The CAP theorem states that a distributed database can only provide two out of three guarantees: consistency, availability, and partition tolerance.

In NoSQL, systems often sacrifice strict consistency to maintain availability and partition tolerance, meaning you may see eventual consistency in replicated data.

3. List 3 advantages and 3 disadvantages of using MongoDB.

Advantages : flexible schema design, high scalability with replication and sharding, and fast reads/writes for big data.

Disadvantages : it may lead to weak consistency issues, lacks native join support, and can consume more memory compared to SQL systems.

4. What are MongoDB collections and documents? Give an example.

In MongoDB, a collection is similar to a table in SQL, and a document is similar to a row but stored in JSON format.

```
Ex : {  
  "name": "Alice",  
  "age": 20,  
  "class": "10A"  
}
```

5. Compare MongoDB's schema flexibility vs relational schema design.

Relational schema design is strict and requires defining columns before inserting data, while MongoDB allows schema flexibility where each document can have different fields. This makes MongoDB better suited for evolving or unstructured data.

6. When would you prefer MongoDB over PostgreSQL? Provide 2 real-world examples.

MongoDB is preferable over PostgreSQL when working with dynamic or unstructured data. For example, in a social media platform storing posts with variable fields or in an IoT application where sensor data formats may vary, MongoDB's schema flexibility is a huge advantage.

7. Explain the difference between `findOne()` and `find()` in MongoDB.

The difference between `findOne()` and `find()` is that `findOne()` returns only the first matching document, while `find()` returns a cursor containing all documents that match the query.

8. Write a MongoDB query to get all users whose age is greater than 25.

```
db.users.find({ age: { $gt: 25 } })
```

9. What are indexes in MongoDB? Why are they important?

Indexes in MongoDB work like in SQL: they are special data structures that improve query performance. Without indexes, MongoDB must scan the entire collection, which is slow. For example, creating an index on name makes searches by name much faster.

10. Debug: A MongoDB query is slow even after adding an index. What should you check?

If a MongoDB query is still slow after adding an index, you should check if the query is actually using the index by running `explain()`. Also ensure that the chosen index field has good selectivity; if too many documents share the same value, the index won't help much. Hardware limitations can also cause slowness.

Section B: Integrating MongoDB with Python (10 Questions)

11. Which Python library is commonly used to interact with MongoDB?

The most commonly used Python library to interact with MongoDB is PyMongo, while MongoEngine is an ODM that provides ORM-like features.

12. Write Python code to connect to MongoDB using pymongo.

```
from pymongo import MongoClient  
client = MongoClient("mongodb://localhost:27017/")
```

```
db = client.school
```

13. How do you insert a new document into the students collection using Python?

```
db.students.insert_one({"name": "Bob", "age": 18})
```

14. Write Python code to update a user's email where name="Alice".

```
db.users.update_one({"name": "Alice"}, {"$set": {"email": "alice@gmail.com"}})
```

15. How do you delete all records from a logs collection using Python?

```
db.logs.delete_many({})
```

16. Explain the difference between insert_one() and insert_many().

The difference between insert_one() and insert_many() is that the first adds a single document while the second allows inserting multiple documents in a single call.

17. Debug: Your Flask app throws ServerSelectionTimeoutError. What might be wrong in MongoDB connection?

If you get a ServerSelectionTimeoutError, it usually means the MongoDB server is not running, the connection string is wrong, or there is a firewall/network issue blocking access.

18. How do you paginate MongoDB results in Python?

Pagination in MongoDB can be done using .skip() and .limit(). For example, to get the 2nd page of 5 results you would skip the first 5 and limit the next 5.

19. Show Python code to aggregate all orders and calculate total revenue.

Pagination in MongoDB can be done using .skip() and .limit(). For example, to get the 2nd page of 5 results you would skip the first 5 and limit the next 5.

20. Which ORM-like Python library provides MongoDB ODM (Object Document Mapper) support?

An ODM-style library for MongoDB in Python is MongoEngine, which works like Django's ORM but for MongoDB.

Section C: Designing Data Persistence with NoSQL (8 Questions)

21. How do you model a **1-to-many relationship** in MongoDB? (e.g., user → posts).

A one-to-many relationship like user → posts can be modeled in MongoDB either by embedding the posts directly inside the user document, or by referencing posts in a separate collection with the user's ID.

22. Compare embedding vs referencing in MongoDB schema design.

Embedding is good for small, tightly related data because it allows faster reads. Referencing is better when related data grows large or needs to be accessed independently, since it avoids duplication and bloated documents.

23. Which approach would you use for storing product reviews: embedded documents or references? Why?

For product reviews, referencing is usually better because reviews can grow to thousands per product and are often managed separately. Embedding them all would make the product document too large.

24. How do you ensure **data consistency** in a distributed MongoDB setup?

To maintain consistency in a distributed MongoDB setup, you can use replica sets, configure appropriate write concern and read concern, and ensure transactions are enabled where needed.

25. Explain sharding in MongoDB. When is it needed?

Sharding in MongoDB means splitting a collection across multiple servers. It becomes necessary when your data set grows so large that a single server cannot handle storage or query load.

26. How do you implement soft deletes in MongoDB?

Soft deletes in MongoDB are done by adding a flag such as `is_deleted: true` instead of physically deleting a document. This allows you to restore or audit deleted records later.

27. What are transactions in MongoDB, and how do they differ from SQL transactions?

Transactions in MongoDB allow multiple operations across documents to execute atomically, similar to SQL transactions. However, they are less common in NoSQL and may have performance trade-offs.

28. Design a MongoDB schema for a **school management system** with Students, Teachers, and Classes.

A school management schema in MongoDB could have separate collections for students, teachers, and classes. Students reference their teacher or class, teachers reference their subjects, and classes store lists of enrolled students.

Section D: Developing User Interfaces with MongoDB (6 Questions)

29. How do you expose MongoDB data via Flask/Django API for frontend consumption?

To expose MongoDB data for a frontend, you typically build an API in Flask or Django. The backend connects to MongoDB using PyMongo, fetches data, and returns it in JSON format, which the frontend can consume.

30. Write a Flask route that fetches all students from MongoDB and returns JSON.

```
from flask import Flask, jsonify
from pymongo import MongoClient
app = Flask(__name__)
db = MongoClient().school
@app.route("/students")
def get_students():
    students = list(db.students.find({}, {"_id": 0}))
    return jsonify(students)
```

31. How would you secure API endpoints that expose MongoDB data?

Securing MongoDB API endpoints involves adding authentication (JWT, sessions, or OAuth) and role-based access control so only authorized users can query sensitive data.

32. Explain how React (or any SPA) consumes data from a Flask/Django backend connected to MongoDB.

In React, data from MongoDB is consumed by making API calls to Flask or Django endpoints. For example, React might fetch data with `fetch("http://api/students")` and then display it in components.

33. What are common pitfalls when directly exposing MongoDB queries to UI?

Exposing MongoDB queries directly to the UI is risky. It can leak too much data, introduce injection vulnerabilities, or cause performance issues by returning massive datasets. Always use backend filters.

34. How do you implement server-side filtering for a UI table populated from MongoDB data?

Server-side filtering is done by accepting query parameters from the UI, applying them in the backend query, and returning only the filtered results. For instance, if the UI table requests students in class 10A, the backend query applies `{class: "10A"}` before returning data.

Section E: Implementing NoSQL Persistence Layer (6 Questions)

35. What is a persistence layer, and why is it needed in software design?

A persistence layer is an abstraction between your application logic and the database. It hides raw queries and provides clean methods to save, update, and fetch data, making your app more modular and easier to test.

36. Show an example of Python code where MongoDB persistence layer is abstracted via a repository pattern.

```
class UserRepository:
    def __init__(self, db):
        self.collection = db.users

    def get_by_id(self, uid):
        return self.collection.find_one({"_id": uid})
```

37. How do you handle schema migrations in MongoDB projects?

MongoDB doesn't have schema migrations like SQL. Instead, migrations are handled through scripts that update documents in batches, or by using tools such as Mongock or custom migration frameworks.

38. Write a Python persistence layer function `get_active_users()` that fetches users with status="active".

```
def get_active_users(db):
    return list(db.users.find({"status": "active"}))
```

39. How do you test MongoDB persistence functions in Python applications?

To test persistence functions, you can either use an in-memory mock database like mongomock, or spin up a temporary MongoDB instance in Docker specifically for tests.

40. In a microservices project, explain how each service should handle its own MongoDB database for better isolation.

In microservices, each service should have its own MongoDB database to ensure isolation and independence. For example, the Student Service would have `students_db`, the Teacher Service would have `teachers_db`, and they would communicate only via APIs, not a shared database.